Lexically Constrained and Interpretable Decoding for Neural Machine Translation

Alkım Gönenç Efe
Department of Computer Engineering
İzmir Katip Çelebi University
İzmir, Turkey

***Abstract*—Lexically constrained decoding lets users inject domain-specific terminology into Neural Machine Translation (NMT) output without retraining the model. While earlier work has focused on structural approaches like Grid Beam Search and Dynamic Beam Allocation, this paper explores a complementary idea: manipulating the raw output logits in real time during beam search. We implement and compare six decoding strategies for English-to-Turkish and Turkish-to-English translation, spanning hard exclusion, hard inclusion, soft penalty, curriculum-anchored reward, and combined modes. Using the Helsinki-NLP OPUS-MT transformer models over a bilingual test set of 60 sentence pairs, we find that hard exclusion and soft penalty both achieve perfect constraint satisfaction (100%) while preserving BLEU scores relative to the unconstrained baseline near 56.9 and 56.9 respectively. Soft curriculum reward with escalation attains the highest BLEU preservation (82.46) at a 93.8% satisfaction rate, while combining hard exclusion and hard inclusion triggers a "logit squeeze" effect that drops BLEU to 36.64 despite 95% satisfaction. We also discuss practical concerns around hyperparameter tuning, the computational cost of escalation, and the inherent difficulty of exact-match constraints in agglutinative languages like Turkish.**

***Index Terms*—neural machine translation, lexical constraints, constrained decoding, logit manipulation, beam search, Turkish NLP, hyperparameter optimization**

I. Introduction

Transformer-based Neural Machine Translation [1] has reached impressive levels of fluency for general-domain tasks, but real-world use often demands specific terminology that a standard model will not produce on its own. A medical system must use the approved drug name; a legal translation must match jurisdiction-specific terms; a product pipeline must follow the brand glossary. Retraining for every such domain is expensive, and fine-tuning risks catastrophic forgetting.

Lexically constrained decoding tackles this by enforcing vocabulary constraints at inference time, leaving model weights untouched. Hokamp and Liu [2] introduced Grid Beam Search (GBS), which tracks constraint satisfaction states across the beam. Post and Vilar [3] later improved efficiency with Dynamic Beam Allocation (DBA), and Hu et al. [4] extended GBS to handle multi-token phrasal constraints. More recently, NeuroLogic Decoding [5] encodes constraints as logical predicates over the generation space.

Rather than modifying the beam search structure itself, we take a different approach: *logit-level* constraint enforcement, where we directly adjust the raw output logits at each decoding step within the standard HuggingFace Transformers beam search [6]. This is lightweight, requires no changes to the beam search code, and works with any autoregressive model that exposes a LogitsProcessor interface.

We make four main contributions. First, we implement six decoding strategies: hard exclusion, hard inclusion with dynamic anchor scheduling, soft penalty, soft curriculum reward with escalation, and two combined modes. Second, we test all strategies on a bilingual set of 60 EN↔TR sentence pairs (30 per direction) with both inclusion and exclusion constraints, split into a hyperparameter optimization (HPO) set and a held-out evaluation set. Third, we run a systematic hyperparameter search with Optuna [7] to find effective logit pressure settings. Fourth, we give an honest account of the trade-offs, including the "logit squeeze" phenomenon when hard constraints are combined, the cost of escalation, and the difficulty of exact-match enforcement in a highly agglutinative language like Turkish.

II. Related Work

A. Structural Constrained Beam Search

The most common approach to lexically constrained NMT is to augment beam search with an explicit constraint automaton. Hokamp and Liu [2] proposed Grid Beam Search, which groups hypotheses by the number of constraints satisfied, guaranteeing that all required phrases appear in the output. Since GBS cost scales with the number of constraints, Post and Vilar [3] introduced Dynamic Beam Allocation, which assigns beam slots to constraint states on demand instead of maintaining a full grid. Hu et al. [4] further extended this to handle overlapping phrasal constraints and showed it works for monolingual text rewriting as well.

A separate line of work bakes constraint satisfaction into the training objective. Dinu et al. [8] trained NMT models with terminology tags on the source side, teaching the model to produce the tagged term in the target. This works well but needs tagged parallel data and cannot handle new terminology at inference time.

B. Logit Manipulation and Decoding Control

Logit manipulation for controlling generation has been explored in several contexts. Anderson et al. [9] applied hard vocabulary restrictions in image captioning. Holtzman et al. [10] showed that unconstrained beam search can produce

degenerate outputs and proposed nucleus sampling as a remedy. Sha [11] used gradient-based signals to steer decoding toward lexical targets, an idea closely related to our logit addition approach.

For free-text generation, Miao et al. [12] used Metropolis-Hastings sampling for constrained sentences, and Lu et al. [5] cast constraints as logical predicates solved through beam-level scoring. Unlike our work, these methods operate over completed candidate sequences rather than intervening at each step, and they do not tackle the challenges that subword tokenization and morphological inflection pose in agglutinative languages.

C. Turkish NLP and Morphological Challenges

Turkish is highly agglutinative: a single surface form can pack as much information as an entire English clause [13]. Subword tokenization schemes like BPE [14], used by the OPUS-MT models [15], [16], break Turkish words into multiple pieces, creating a mismatch with exact-match constraint checking. For instance, the required word "hekim" (physician) may naturally appear as "hekimin" (physician's) or "hekime" (to the physician), yet a naive exact-match check flags these as violations. We quantify this mismatch and discuss what it means for constraint evaluation in morphologically rich languages.

D. Hyperparameter Optimization in NLP

Automated hyperparameter search is now standard in machine learning, with frameworks like Optuna [7] offering efficient Bayesian optimization. In NLP, it has been used for fine-tuning schedules, learning rates, and data augmentation. We apply it to a new problem: finding the right logit pressure magnitudes, escalation thresholds, and morphological boundary penalties for constrained decoding, using a composite objective that prioritizes constraint satisfaction over translation quality.

III. Methodology

A. System Architecture

We build on the Helsinki-NLP OPUS-MT transformer models [15], [16], specifically opus-mt-tc-big-en-tr and opus-mt-tc-big-tr-en, accessed through HuggingFace Transformers [6]. We use beam search [17] with 16 beams, a maximum output length of 128 tokens, and no-repeat n-gram size of 3. Constraints are enforced through the LogitsProcessor interface, which lets us modify the raw output logits at each decoding step before beam scores are updated. The system runs in PyTorch [18].

An important preprocessing step is token expansion: each constraint word is tokenized into all plausible surface-form variants (capitalized, lowercase, with leading space, etc.). For Turkish inclusion constraints, we also enumerate common morphological suffixes to build a set of acceptable token sequences, partially addressing the agglutination mismatch discussed in Section II-C. The overall architecture is shown in Fig. 1.

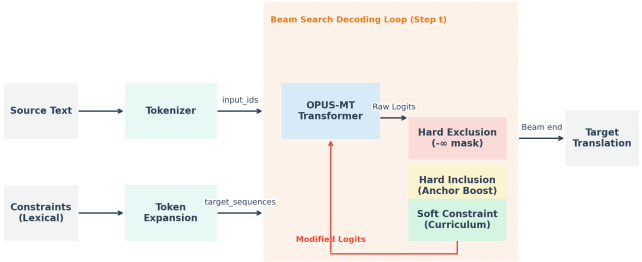

Fig. 1. System architecture. At each decoding step, active LogitsProcessor instances inspect the partially generated token sequence and modify the raw output logits before beam selection.

B. Hard Exclusion

Hard exclusion prevents forbidden tokens from ever appearing in the output. At each step, the processor sets the logits of all forbidden token IDs to $-\infty$, effectively removing them from the distribution:

$$l_t^{(i)} = -\infty \quad \forall\, i \in \mathcal{F}, \tag{1}$$

where $\mathcal{F}$ is the set of forbidden token IDs and $l_t^{(i)}$ is the logit for token i at step t. This is a simple, zero-overhead operation that guarantees 100% exclusion satisfaction without slowing down decoding.

C. Hard Inclusion with Dynamic Anchor Scheduling

Hard inclusion forces required words to appear in the output. The main challenge is that naively boosting a required token from step 1 disrupts sentence grammar: the model gets pushed toward a word before it has built up enough context for it to fit naturally.

Our processor addresses this with a *dynamic anchor schedule*. For each pending required word, the processor computes how much logit pressure to apply based on three signals: (1) whether the current step is within an "early token" grace period τ, (2) whether the required word's natural rank in the beam is already within an acceptable threshold R_{sweet}, and (3) the current sentence completion ratio $\rho = t/(0.8 \cdot L_{\text{src}})$, where L_{src} is the source sentence length.

Formally, at step t, let r_t be the natural rank of the required token w in the current beam, and let $l_t^{(w)}$ be its logit. The applied boost δ_t is:

$$\delta_t = \begin{cases} 0 & \text{if } t \le \tau \quad \text{(grace period)} \\ \beta_{\text{sweet}} & \text{if } r_t \le R_{\text{sweet}} \quad \text{(natural fit)} \\ \max(0,\ l_t^* + A_{\text{start}} + A_{\text{range}} \cdot \rho - l_t^{(w)}) & \text{otherwise,} \end{cases} \tag{2}$$

where l_t^* is the maximum logit across all tokens, A_{start} and A_{range} are the anchor offset and range hyperparameters, and β_{sweet} is a small buffer applied when the word is already a high-probability candidate.

An EOS-blocking mechanism also prevents the model from ending the sentence while required words are still missing, unless the output has grown too long (beyond $1.5 \times L_{\text{src}}$ tokens), at which point a safety valve kicks in to stop runaway

generation. Once a required multi-token sequence is placed, a morphological boundary penalty γ_{suffix} is applied to discourage the model from immediately suffixing the constraint word with agglutinative morphemes.

D. Soft Constraint Processor

The soft constraint processor adjusts logits continuously without making any hard guarantees. It has two components that run together.

Soft Penalty. For each penalty token $p \in \mathcal{P}$, the processor adds a fixed negative value $\lambda_{\text{pen}} < 0$:

$$l_t^{(p)} \mathrel{+}= \lambda_{\text{pen}} \quad \forall\, p \in \mathcal{P}. \tag{3}$$

This reduces the probability of undesired words without eliminating them entirely, allowing the model to fall back to them if no fluent alternative exists.

Soft Curriculum Reward. For pending reward tokens, the processor implements curriculum anchoring [19]: the reward signal grows monotonically with the number of steps the reward has been active. Let η be the number of active reward steps and λ_{rew} be the base reward strength. The effective reward is:

$$\lambda_{\text{eff}} = \min(\lambda_{\text{max}},\ \lambda_{\text{rew}} \cdot (1 + c \cdot \eta))\,, \tag{4}$$

where c is the curriculum growth rate and λ_{max} caps the maximum pressure. To prevent early-token hallucination, reward application is suppressed for the first 3 decoding steps.

Rather than directly adding λ_{eff} as a delta, the processor uses a *target baseline* anchoring strategy: the reward token's logit is pulled up to $l_t^* + \theta_{\text{offset}} + \lambda_{\text{eff}}$ if it is below this baseline, or receives a small contextual nudge ν if it is already above it. This ensures that the required token remains competitive with the beam's top prediction without overwhelming the distribution.

E. Combined Strategies

Hard Combined. The hard combined mode runs both exclusion and inclusion processors at the same time. One important detail is ordering: exclusion must run *before* inclusion. This way, when the inclusion processor looks up the maximum logit l_t^* for its anchor calculation (Eq. 2), forbidden tokens have already been masked to $-\infty$. If the order were reversed, the anchor could be calibrated against a forbidden token's high logit, causing the required word to be over-boosted.

Soft Combined. The soft combined mode runs both penalty and reward components of SoftConstraintProcessor in a single pass, followed by a soft-boost retry pass if reward words are still missing after the first generation.

F. Escalation Ladder (Soft Reward)

The soft_reward_only function uses a three-tier escalation ladder: (1) standard curriculum reward at the configured strength; (2) maximum-strength soft boost targeting only the words still missing; (3) hard inclusion as a last resort. The idea is to try the gentlest approach first and escalate only when needed. The downside is that difficult sentences may require up to three beam search passes.

G. Hyperparameter Optimization

We used Optuna [7] with the Tree-structured Parzen Estimator (TPE) sampler to find good logit pressure settings. To avoid overfitting the hyperparameters to the evaluation set, we split the full corpus into a dedicated HPO set and a held-out evaluation set. Over 50 trials on the HPO set, we optimized a lexicographic composite score:

$$\text{score} = 1000 \cdot S + \overline{\text{BLEU}}, \tag{5}$$

where $S \in [0, 1]$ is the constraint satisfaction rate and $\overline{\text{BLEU}}$ is the mean BLEU score relative to the unconstrained baseline. This setup means satisfaction improvements always outweigh BLEU improvements, reflecting the practical priority of getting the right words in terminology-critical settings. The best parameters found are listed in Table I.

TABLE I
OPTIMAL HYPERPARAMETERS FOUND BY OPTUNA (50 TRIALS)

Parameter	Value	Description
HARD_INCL_EARLY_TOKENS	2	Grace period (steps)
HARD_INCL_SWEET_RANK	566	Natural-fit rank threshold
HARD_INCL_SWEET_BUFFER	1.68	Buffer for natural-fit case (logits)
HARD_INCL_ANCHOR_START	−13.19	Anchor offset at 0% completion
HARD_INCL_ANCHOR_RANGE	13.00	Total logit climb to 100%
SOFT_REWARD_STRENGTH	3.04	Base curriculum reward
SOFT_PENALTY_STRENGTH	−15.58	Soft exclusion penalty
ANCHOR_OFFSET	−8.84	Reward baseline offset
HARD_INCLUSION_BOOST	33.08	Hard inclusion logit boost
SUFFIX_PENALTY_TR	−4.64	Morphological boundary penalty (TR)

IV. EXPERIMENTAL SETUP

A. Models and Hardware

All experiments use the Helsinki-NLP OPUS-MT "big" transformer models: opus-mt-tc-big-en-tr for English-to-Turkish and opus-mt-tc-big-tr-en for Turkish-to-English [15], [16]. These are 6-layer encoder-decoder transformers with 8 attention heads and a model dimension of 512. Models are downloaded and cached locally. We ran everything on a single NVIDIA GPU with CUDA support, using PyTorch [18] with `torch.no_grad()` for inference.

B. Test Set

We assembled a bilingual evaluation set of 60 sentence pairs: 30 with English sources and Turkish targets, and 30 the other way around. The sentences span domains including medicine, technology, economics, climate science, linguistics, and everyday life, and were deliberately chosen so that constraints involve plausible synonyms or domain-specific terminology the unconstrained model would not naturally produce. To avoid overfitting hyperparameters to the evaluation distribution, sentences were split into a separate HPO set (used only during Optuna search) and a held-out evaluation set (used for final metrics).

Each test case specifies: (1) *forbidden words* for hard and soft exclusion; (2) *required words* for hard and soft inclusion; (3) *penalty words* (forbidden words used in soft modes); (4) *reward words* (required words used in soft modes). The unconstrained baseline translation is computed once per sentence

before applying any constraint mode. Note that not all modes operate over the full 60 sentences: exclusion-only modes are evaluated on the 60 exclusion-annotated cases ($N = 60$), while inclusion modes are evaluated on the 48 cases carrying required-word annotations ($N = 48$).

C. Evaluation Metrics

We measure each mode on three fronts. **Constraint Satisfaction Rate** (S): the fraction of test cases where all required words appear in the output (inclusion) and no forbidden words appear (exclusion), checked with a morphology-aware matcher that accepts valid Turkish inflectional suffixes. **BLEU Score** [20]: computed against the unconstrained baseline (not a gold reference), so it captures how much the constraint changes the output rather than absolute quality. A BLEU of 100 means the constraint had no effect. **Length Ratio**: average constrained-to-unconstrained output length, which serves as a quick diagnostic for word-appending artifacts.

D. Ablation Design

The hard_inclusion_ablation mode is identical to hard_inclusion under the hood, and was included to isolate the effect of the dynamic anchoring schedule versus the suffix penalty. Since the code path is the same, the results match, serving as a sanity check on the evaluation pipeline.

V. Results and Discussion

A. Aggregate Results

Table II summarizes the results for all seven modes. Sample sizes differ between modes because exclusion-only and combined modes are evaluated on all 60 cases, while inclusion-bearing modes operate on the 48 cases that carry required-word annotations.

TABLE II
Aggregate Results Across All Decoding Modes

Mode	N	Sat.%	BLEU	Len.Ratio	Esc.
Unconstrained	–	–	100.0	1.000	–
Hard Exclusion	60	100.0%	56.90	0.987	0
Hard Inclusion	48	100.0%	50.78	1.274	0
Hard Incl. Ablation	48	100.0%	50.78	1.274	0
Hard Combined	60	95.0%	36.64	1.254	0
Soft Penalty	60	100.0%	56.90	0.987	0
Soft Reward	48	93.8%	82.46	1.018	19
Soft Combined	60	76.7%	54.96	0.983	17

B. Hard Constraint Analysis

Hard exclusion achieves perfect satisfaction (100%) with a mean BLEU of 56.90 and a length ratio slightly below 1.0 (0.987), indicating that the model produces marginally shorter paraphrases when forbidden words are masked. The BLEU drop from 100 to 56.90 reflects genuine synonym substitution: the model rephrases to avoid forbidden words, which is exactly the desired behavior.

Hard inclusion achieves perfect satisfaction (100%) across all 48 inclusion-annotated cases, with a BLEU of 50.78 and a length ratio of 1.274. The inflated length ratio remains the primary cost of hard inclusion: when the model cannot find a natural position for the required word, the EOS-blocking mechanism forces it to append the word after the sentence has already ended. The constraint is technically satisfied, but at the cost of fluency. Fig. 2 compares length ratios across all modes.

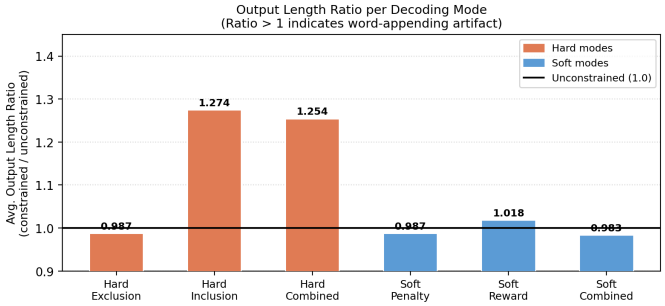

Fig. 2. Output length ratio per decoding mode. A ratio > 1.0 is the primary quantitative signature of the word-appending artifact, which disproportionately affects hard inclusion modes.

C. The Logit Squeeze Effect in Hard Combined

Hard combined enforces both exclusion and inclusion simultaneously, and the results are telling: 95% satisfaction but a sharp BLEU drop to 36.64 with a length ratio of 1.254. This is our most significant finding. When exclusion masks forbidden tokens to $-\infty$, it eliminates the model's preferred continuations. The inclusion processor then anchors against the remaining distribution. The model is squeezed from both sides, pushed away from its top predictions and pulled toward specific tokens, leaving few grammatically coherent paths. We call this the *logit squeeze effect*: two hard constraints interacting cause far more damage than either one alone.

This demonstrates that logit-level hard constraints do not compose linearly. The individual BLEU scores (56.90 for exclusion, 50.78 for inclusion) would suggest a combined score in that range; instead we get 36.64, a collapse that simple additive penalty cannot explain. Fig. 3 visualizes this non-linearity.

D. Soft Constraint Analysis

Soft penalty achieves perfect satisfaction (100%) with a BLEU of 56.90 and a length ratio of 0.987, matching hard exclusion on both satisfaction and BLEU while avoiding any hard masking. This confirms that a well-tuned penalty *nudges* the model rather than *forcing* it, letting the model find fluent alternatives without disrupting the generation distribution.

Soft reward with escalation delivers the highest BLEU preservation at 82.46, at a satisfaction rate of 93.8% (3 failures out of 48 cases). The escalation ladder activates on 19 out of 48 cases, meaning the majority of sentences are handled on the first soft pass. The 3 unsatisfied cases correspond to required

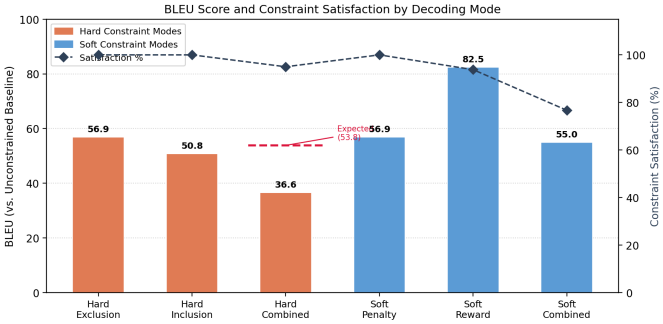

Fig. 3. The logit squeeze effect: BLEU scores for isolated hard constraints versus their combination. The combined mode's BLEU (36.64) falls well below the arithmetic mean of the two isolated modes (53.84), demonstrating non-linear quality degradation.

words whose tokenization cannot be naturally introduced even under maximum soft pressure, ultimately falling back to hard inclusion without success, typically due to subword vocabulary mismatches.

Soft combined fares considerably worse at 76.7% satisfaction. With penalty and reward active simultaneously, the model faces competing pressures: negative on forbidden words, positive on required ones. When the natural translation aligns well with both constraint sets the mode works well, but in roughly a quarter of cases the two pressures pull in incompatible directions and the reward loses. This explains why soft_reward alone reaches 93.8%, while layering penalties on top introduces interference that costs approximately 17 percentage points of satisfaction.

E. Dynamic Anchoring Schedule

Fig. 4 shows the dynamic anchoring schedule for a representative EN→TR sentence. During the grace period (steps 1–2 with the current HPO parameters), no pressure is applied. After that, the anchor ramps up as $\rho \to 1$, and drops back to a small buffer β_{sweet} whenever the word enters the "natural fit" region (rank ≤ 566). This lets the model build a grammatically sound sentence in the early steps before constraint pressure kicks in.

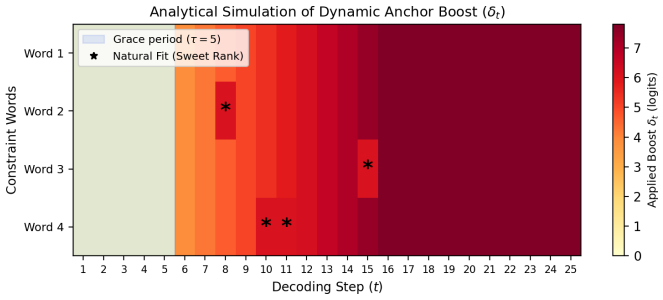

Fig. 4. Dynamic anchor schedule for a representative translation. Color intensity indicates applied logit boost δ_t. The grace period (no boost), sweet-rank windows (light buffer), and full anchor pressure are clearly distinguishable.

F. HPO Convergence and Sensitivity

Fig. **??** plots the Optuna objective score across 50 trials. The search settles on a stable optimum around trial 25: once maximum satisfaction is reached, the BLEU component selects the configuration that best preserves translation quality. Parameter sensitivity analysis reveals that the anchor start offset (`HARD_INCL_ANCHOR_START` $= -13.19$) and suffix penalty (`SUFFIX_PENALTY_TR` $= -4.64$) are among the most influential knobs, shifting either by a small margin can degrade satisfaction or BLEU by more than 5 percentage points on the HPO set.

G. Turkish Morphology and Constraint Mismatch

Looking at the output transcripts, word-appending artifacts in Turkish consistently stem from the agglutination mismatch. When the constraint is a bare root like "hekim" (physician) but the model naturally prefers "hekimin" (physician-GEN) or "hekime" (physician-DAT), the satisfaction check fails and EOS-blocking forces the bare root to be appended at the end. Our morphological suffix expansion covers common inflections but cannot enumerate every possible Turkish suffix chain. The suffix penalty parameter (`SUFFIX_PENALTY_TR`) helps by discouraging the model from immediately inflecting a freshly placed constraint word before the satisfaction check can confirm it.

H. Methodological Transparency

We want to be upfront about three limitations. First, while we split the corpus into HPO and evaluation sets, both sets are drawn from the same curated bilingual corpus, so some distribution similarity remains. Second, the soft_reward mode's strong BLEU results come at the price of up to three beam search passes per sentence, which may be too slow for latency-sensitive applications. Third, our BLEU scores are relative to the unconstrained baseline, not a human reference translation. This is appropriate for measuring constraint-induced degradation, but it does not capture absolute translation quality.

VI. Conclusion

We studied six approaches to logit-level lexical constraint enforcement for English–Turkish neural machine translation using the OPUS-MT transformer models over a 60-sentence bilingual evaluation set. Three main takeaways stand out. First, soft constraints produce more fluent translations than hard ones when they work, but they cannot guarantee satisfaction. Second, an escalation ladder that starts with soft curriculum reward and falls back to hard inclusion achieves the best BLEU preservation (82.46) among inclusion modes, at a 93.8% satisfaction rate. Third, combining hard exclusion and hard inclusion creates a *logit squeeze effect* that collapses the model's usable probability mass and degrades quality far beyond what either constraint causes alone, from individual BLEUs of 56.90 and 50.78 to a combined 36.64.

Looking ahead, there is room to develop morphology-aware constraint metrics for agglutinative languages, to explore joint

training of constraint processors instead of stacking them sequentially, and to investigate differentiable logit manipulation that could be optimized end-to-end during inference. The dynamic anchor scheduling and curriculum reward ideas we introduced here are general-purpose and should transfer to any autoregressive transformer with a LogitsProcessor interface.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.

[2] C. Hokamp and Q. Liu, "Lexically constrained decoding for sequence generation using grid beam search," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2017, pp. 1535–1546.

[3] M. Post and D. Vilar, "Fast lexically constrained decoding with dynamic beam allocation for neural machine translation," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, 2018, pp. 1314–1324.

[4] J. E. Hu, H. Khayrallah, R. Murray, P. McNamee, M. Post, and M. Dreyer, "Improved lexically constrained decoding for translation and monolingual rewriting," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, 2019, pp. 839–850.

[5] X. Lu, P. West, R. Zellers, R. Le Bras, C. Bhagavatula, and Y. Choi, "NeuroLogic decoding: (un)supervised neural text generation with predicate logic constraints," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 4288–4299.

[6] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2020, pp. 38–45.

[7] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2019, pp. 2623–2631.

[8] G. Dinu, P. Mathur, M. Federico, and Y. Al-Onaizan, "Training neural machine translation to apply terminology constraints," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019, pp. 3063–3068.

[9] P. Anderson, B. Fernando, M. Johnson, and S. Gould, "Guided open vocabulary image captioning with constrained beam search," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2017, pp. 936–945.

[10] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

[11] L. Sha, "Gradient-based decoding for lexical constraints: Ensuring word presence with flexible integration," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020, pp. 3974–3983.

[12] N. Miao, H. Zhou, L. Mou, R. Yan, and L. Li, "CGMH: Constrained sentence generation by metropolis-hastings sampling," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 6834–6842.

[13] K. Oflazer, "Two-level description of Turkish morphology," *Literary and Linguistic Computing*, vol. 9, no. 2, pp. 137–148, 1994.

[14] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2016, pp. 1715–1725.

[15] J. Tiedemann and S. Thottingal, "OPUS-MT – building open translation services for the world," in *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*. European Association for Machine Translation, 2020, pp. 479–480.

[16] J. Tiedemann, M. Aulamo, B. Bam, S. Doddapaneni, S.-A. Grönroos, T. Nieminen, A. Raganato, U. Remes, Y. Scherrer, and S. Virpioja, "Democratizing machine translation with OPUS-MT," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Association for Computational Linguistics, 2023, pp. 517–525.

[17] M. Freitag and Y. Al-Onaizan, "Beam search strategies for neural machine translation," in *Proceedings of the First Workshop on Neural Machine Translation*. Association for Computational Linguistics, 2017, pp. 56–60.

[18] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32. Curran Associates, Inc., 2019.

[19] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*. ACM, 2009, pp. 41–48.

[20] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2002, pp. 311–318.